



Zadanie 2

Quick sort $O(n \log n)$

Heap sort $O(n \log n)$

Osobné číslo: 121834

1 Zadanie

Porovnanie efektívnosti dvoch zvolených algoritmov triedenia. Zhodnotenie, či sa teoretická zložitosť rovná empirickým výsledkom.

Porovnávané algoritmy:

- Quick sort - $O(n \log n)$
- Heap sort - $O(n \log n)$

2 Analýza metód triedenia

Pri triediacich algoritmoch si všímame ich vlastnosti a na základe nich ich porovnávame. Najdôležitejšou vlastnosťou je, koľko času na usporiadanie prvkov spotrebujú. Samotné algoritmy sa delia podľa relatívneho času, tzv. O- notáciou, čo je asymptotické ohraničenie zhora.

Quick sort

Triediaci algoritmus Quick sort patrí medzi najrýchlejšie triediace algoritmy. Jeho časová zložitosť je závislá od správnej voľby pivotu. V najhoršom prípade, ak by sme zvolili ako pivot najmenší, alebo najväčší prvok poľa, môže byť jeho časová zložitosť - $O(n^2)$. Preto ak nemáme informácie o zoradení prvkov v poli, zvolíme ako pivot prostredný prvok a tým znížime pravdepodobnosť najhoršieho prípadu. V tom prípade bude jeho priemerná časová zložitosť - $O(n \log n)$.

Pri implementácii algoritmu funkcie *QuickSort* na zoradenie poľa (v našom prípade je to pole náhodne generovaných celých čísel) voláme funkciu so vstupnými parametrami *data[]* (pole ktoré sa má zotriediť), *left* (prvý prvok poľa), *right* (posledný prvok poľa).

Vo funkcii si zvolíme *pivot*, čo bude prostredný prvok poľa, a rozdelíme pole na dve časti. Presunieme všetky menšie prvky od pivotu na ľavú stranu a všetky väčšie prvky na pravú stranu od pivotu. Následne funkciu voláme rekurzívne pre ľavú a pravú časť poľa na zotriedenie prvkov menších ako *pivot* a väčších ako *pivot*. Toto opakujeme až kým je celé pole zotriedené.

Heap sort

Heap sort je technika triedenia založená na porovnávaní, pri ktorej najprv vytvoríme maximálnu haldy a potom zameníme koreňový prvok s posledným prvkom (krát veľkosť) a zakaždým zachováme vlastnosť haldy, aby sme ju nakoniec zoradili.

Funkcia *Heapify* opravuje haldy na konkrétnom uzle. Vstupné parametre sú pole celých čísel (*data*), *N* je počet prvkov v poli a *i* je index uzla, ktorý sa má opraviť.

Prvý krok funkcie je inicializácia premennej *maximum* na index uzla, ktorý sa má opraviť. Následne sa určia indexy jeho ľavého a pravého syna, ktoré sa vypočítajú podľa vzorca $left = 2 * i + 1$ a $right = 2 * i + 2$.

Potom sa overí, či existuje ľavý syn a či je jeho hodnota väčšia ako hodnota uzla s indexom *maximum*. Ak áno, premenná *maximum* sa aktualizuje na index ľavého syna. Podobne sa overí, či existuje pravý syn a či je jeho hodnota väčšia ako hodnota uzla s indexom *maximum*. Ak áno, premenná *maximum* sa aktualizuje na index pravého syna.

Ak sa po vykonaní týchto podmienok zistí, že index uzla *i* sa nezhoduje s indexom uzla *maximum*, tak sa výmenou hodnôt uzlov na indexoch *i* a *maximum* dosiahne oprava haldy na

uzle s indexom *maximum*. Následne sa funkcia rekurzívne volá na uzol s indexom *maximum*, aby sa zabezpečila korektná halda.

Funkcia *HeapSort* slúži na triedenie dátového poľa data pomocou algoritmu Heap sort. V prvom kroku sa vytvorí halda pomocou volania funkcie *Heapify* pre každý uzol, ktorý má syna, od posledného uzla až po koreň (od indexu $N/2 - 1$ po index 0).

V druhom kroku sa postupne vyberajú prvky z haldy v poradí od najväčšieho po najmenší. Pri každom vybraní sa najväčší prvok (na indexe 0) vymení s posledným prvkom a použije sa funkcia *Heapify*, aby sa zabezpečila korektná halda na zostávajúcich prvok. To sa opakuje, kým sa nevyberú všetky prvky z haldy a dátové pole bude zotriedené.

Časová zložitosť funkcie *Heapify* je $O(\log n)$. Časová zložitosť funkcie *HeapSort* je $O(n \log n)$, pretože tu na $n/2$ prvkov zavoláme funkciu *Heapify*. Samotné triedenie sa skladá z vybudovania haldy a n násobného volania funkcie *Heapify*, teda celkový čas triedenia je $O(n \log n) + n * O(\log n) = O(n \log n)$.

3 Zdrojové kódy

Funkcia QuickSort

```
void QuickSort(int data[],int left, int right)
{
    if(left < right)
    {
        int i = left, j = right;
        int pivot = data[(left + right)/2 ];
        do
        {
            while (data[i] < pivot) i++;
            while (pivot < data[j]) j--;
            if (i <= j)
            {
                swap(data[i], data[j]);
                i++;
                j--;
            }
        } while (i <= j);
        QuickSort(data, left, j);
        QuickSort(data, i, right);
    }
}
```

Funkcia Heapify

```
void Heapify(int data[], int N, int i)
{
    int maximum = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < N && data[left] > data[maximum])
```

```
{
    maximum = left;
}
if(right < N && data[right] > data[maximum])
{
    maximum = right;
}
if(maximum != i)
{
    swap(data[i], data[maximum]);
    Heapify(data, N, maximum);
}
}
```

Funkcia HeapSort

```
void HeapSort(int data[], int N)
{
    for(int i = N/2 - 1; i >= 0; i--)
    {
        Heapify(data, N, i);
    }
    for(int i = N - 1; i > 0; i--)
    {
        swap(data[0], data[i]);
        Heapify(data, i, 0);
    }
}
```

Funkcia Check

```
\\ funkcia na overenie spravnosti zoradenia pola
bool Check(int data[], int n)
{
    for(int i = 0; i < n - 2; i++)
    {
        if(data[i+1] < data[i])
            return false;
    }
    return true;
}
```

Funkcia main()

```
int main()
{
    int n;
    for(int j = 1; j <= 10; j++)
    {
        n = j * 1000000;
        int *arr_Quick = new int[n];
        int *arr_Heap = new int[n];

        srand(time(NULL));
    }
}
```

```
double timeT_Quick = 0, timeT_Heap = 0;
for(int i = 0; i < 3; i++)
{
    for(int k=0; k < n; k++)
    {
        *(arr_Quick+j) = rand();
        *(arr_Heap+j) = *(arr_Quick+j);
    }
    clock_t t;
    t = clock();
    QuickSort(arr_Quick, 0, n-1);
    t = clock() - t;
    double tot_time = ((double)t)/CLOCKS_PER_SEC;
    timeT_Quick += tot_time;

    bool check_Quick = Check(arr_Quick, n);

    cout<<i+1<<". meranie QUICKSORT pre velkost pola "<<n<<": "<<
        tot_time<<" s"<<endl;
    if(check_Quick)
    {
        cout<<"Zoradenie OK";
    }
    else
    {
        cout<<"ERORR";
    }
    cout<<endl;
    t = clock();
    HeapSort(arr_Heap, n);
    t = clock() - t;
    tot_time = ((double)t)/CLOCKS_PER_SEC;
    timeT_Heap += tot_time;

    bool check_Heap = Check(arr_Heap, n);

    cout<<i+1<<". meranie HEAPSORT pre velkost pola "<<n<<": "<<
        tot_time<<" s"<<endl;
    if(check_Heap)
    {
        cout<<"Zoradenie OK";
    }
    else
    {
        cout<<"ERORR";
    }
    cout<<endl;
}
cout<<"Priemerny cas QUICKSORT: "<<timeT_Quick/3<<endl;
cout<<"Priemerny cas HEAPSORT: "<<timeT_Heap/3<<endl;
cout<<endl;
delete [] arr_Quick;
delete [] arr_Heap;
}

return 0;
}
```

4 Overenie triediacich algoritmov

Tabuľky časov triediacich algoritmov

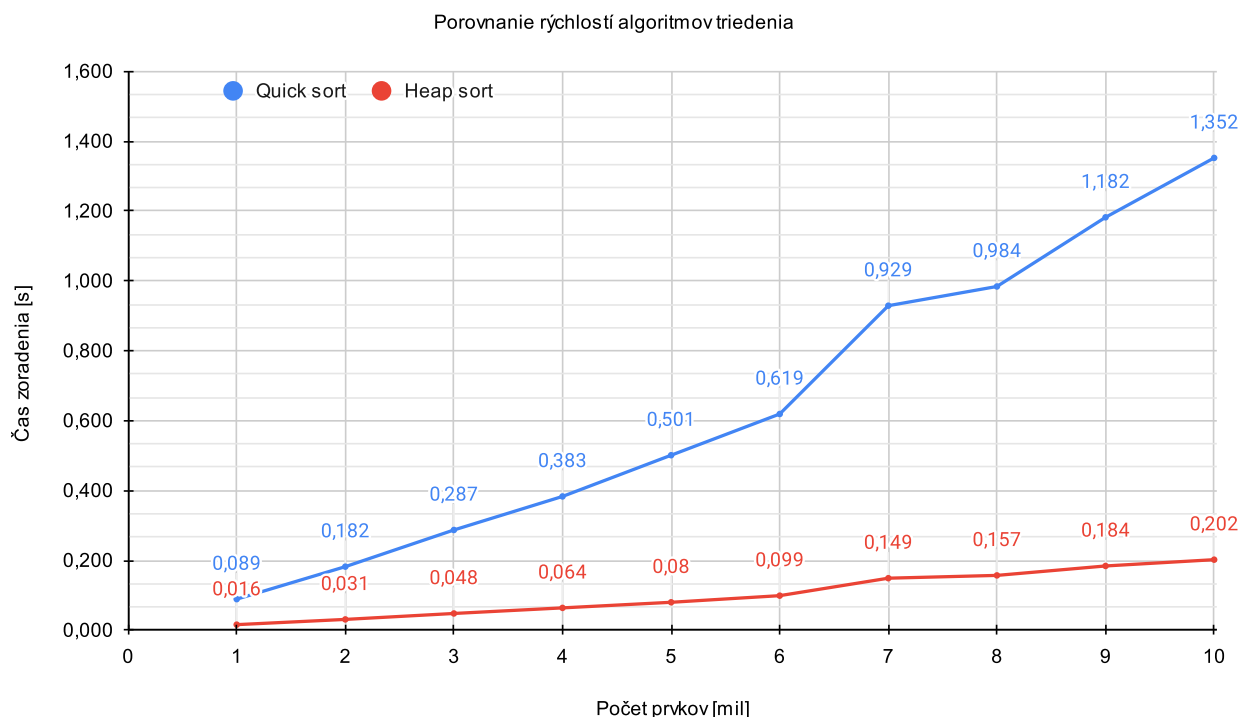
1. algoritmus Quick sort

n [mil]	1	2	3	4	5	6	7	8	9	10
séria										
Mer. 1 [s]	0,091	0,184	0,287	0,385	0,503	0,638	0,874	1,018	1,116	1,346
Mer. 2 [s]	0,088	0,18	0,285	0,385	0,498	0,61	0,917	0,994	1,299	1,286
Mer. 3 [s]	0,087	0,183	0,289	0,38	0,502	0,608	0,996	0,949	1,131	1,423
Čas behu [s]	0,089	0,182	0,287	0,383	0,501	0,619	0,929	0,987	1,182	1,352

2. algoritmus Heap sort

n [mil]	1	2	3	4	5	6	7	8	9	10
séria										
Mer. 1 [s]	0,017	0,033	0,051	0,067	0,084	0,11	0,159	0,163	0,214	0,216
Mer. 2 [s]	0,016	0,03	0,047	0,062	0,078	0,092	0,14	0,153	0,175	0,195
Mer. 3 [s]	0,015	0,031	0,046	0,062	0,078	0,094	0,148	0,153	0,164	0,194
Čas behu [s]	0,016	0,031	0,048	0,064	0,08	0,099	0,149	0,157	0,184	0,202

5 Zhodnotenie triediacich algoritmov



6 Záver

Na základe meraní som zistil, že čas potrebný na zoradenie je závislý od veľkosti vstupného poľa. Namerané hodnoty meraných algoritmov triedenia znázornené v grafe sa približujú k teoretickým hodnotám týchto algoritmov. Môj graf sa zhoduje tvarom s grafom zložitosti $O(n \log n)$.

Mierne odchýlky mohli byť spôsobené usporiadaním hodnôt vo stupnom poli, ako aj výberom pivota, behom iných programov počas merania, alebo parametrami zariadenia na ktorom boli merania vykonávané.

Tabuľka parametrov PC

Parameter	Hodnota
CPU	Intel(R) Core(TM) i5-8250U, 1.80 GHz
RAM	16 GB
Kompilátor/ vývojové prostredie	gcc / CodeBlocks
Operačný systém	Windows 10 HOME

Vytvorené v L^AT_EX